Overview

Open Source Projects and their Effect on Software Development

Communities, Collaboration and Impact

Overview

What we'll explore

- What open source really means today
- How communities form and sustain projects
- Projects that changed entire fields
- The lasting impact on the software industry

1.1 What open source really means

What open source really means today is less about “you can see the code” and more about a specific set of freedoms that **The Open Source Initiative** (and the Free Software movement before it) defined.

> *The Open Source Initiative (OSI) is a global non-profit organization founded in 1998, serving as the guardian and standard-setting body for the "Open Source Definition" . In simple terms, it defines what counts as "open source" and which licenses are legitimate.*

True open source gives you the right to use the software for any purpose, study how it works, modify it, and redistribute both the original and **your modified versions** (including commercially) under the same license terms. **The source code being public is necessary, but it is not sufficient.** That is why the distinctions matter.

Open Source

Open Source means the entire project lives under an OSI-approved license such as the GPL. **Anyone can download it, change it, ship it in a commercial product, or fork it.** The only real obligation under the GPL is that **if you distribute a modified version, you must also make the source available under the same terms.**

Source Available

Source Available is the category Unreal Engine sits in. Epic publishes the full C++ source on GitHub. You can read it, debug it, and modify it for your own projects. That transparency is extremely valuable. But **the license is Epic's EULA, not an open-source license.** You cannot freely redistribute the engine code, and once your product's lifetime revenue crosses $1 million you owe a 5% royalty to Epic Games company. **The source is available; the freedoms are not.**

Open Core

Open Core is the hybrid model many commercial companies use. The "core" of the product is released under a genuine open-source license so the community can use, modify, and contribute to it. **Additional features (usually the ones enterprises actually need like advanced security, scaling, support tooling, marketplace integrations, etc.) remain proprietary and require a paid license. VS Code is a well-known case of this pattern**. Microsoft publishes the underlying Code/Project under the **MIT license**. The branded VS Code binary you download from Microsoft's site, however, ships under Microsoft's own license. **The open-source core exists and is real; the product most people actually run is not fully open**.

The practical difference is simple:

open source maximizes user freedom and community contribution; source available maximizes transparency while keeping commercial control; open core tries to get the best of both by giving away the foundation and selling the extras.

In short, "open source" today is a legal and philosophical claim about rights, not just a marketing phrase about visible code. **When the license does not grant the full set of freedoms, it's not really Open Source, even if the source is completely public.**

How communities form and sustain projects

In practice, the project almost always comes first. Someone (or a small team) builds something useful to solve their own problem. **Linus Torvalds** wrote the first Linux kernel because he wanted a free Unix-like system for his own machine. **Ton Roosendaal** started Blender as an in-house tool for his animation studio. **Guido van Rossum** created Python because he needed a better scripting language. **Once that early version exists and is made available under an open license, people who share the same problem start using it**. A subset of those users eventually become contributors. Fixing bugs, writing documentations, or adding features they themselves need. That is how the community forms around a working piece of software.

Sustaining these projects is a different challenge. Pure volunteer energy works for a surprisingly long time, especially in the early years when passion and the joy of solving hard problems are high. But as a project grows and becomes critical infrastructure,

pure volunteerism is rarely enough. The successful long-lived projects develop mixed models.

Many rely on foundations (Blender Foundation, Linux Foundation, Apache Software Foundation) that provide neutral governance, legal protection, and a way to collect and distribute money without any single company owning the project. Funding comes from a combination of individual donations, corporate sponsorships and a variety of other ways. Companies that depend on the software often pay developers to work on it full-time — either by employing maintainers directly or by contributing through the foundation. In some cases the project itself generates revenue through open-core features, dual licensing, or professional services, while keeping the core free.

What keeps people coming back is not only money. Shared purpose, public recognition, the ability to shape something important, and the career value of being a known contributor all play major roles. Good governance (clear decision-making, codes of conduct, mentorship for newcomers) reduces burnout and conflict. The healthiest communities treat community work itself (documentation, triage, welcoming new contributors) as first-class contributions, not just an afterthought to the code.

So the pattern is usually:

1. useful software attracts users
2. some users become contributors
3. a community forms and, if it is lucky and deliberate, builds the structures (funding, governance, culture) that let the project outlive any single person or company.

The community does not create the project, it inherits and then continually reinvents it.

Projects that changed entire fields

Here are six notable projects, from the infrastructure everyone already knows to a few that quietly rewrote the rules in creative and creative-adjacent domains.

Linux and its major distributions

Before Linux, serious multi-user operating systems were expensive and proprietary. Linus Torvalds's kernel, combined with the GNU tools and later the big distributions (Debian, Red Hat, Ubuntu, and others), gave the world a free, high-quality Unix-like system that anyone could run, modify, and ship. The lasting effect is hard to overstate. Linux now runs most of the internet's servers, the majority of supercomputers, Android devices, and a growing share of desktops and embedded systems. It also proved that a purely open, volunteer-heavy model could produce infrastructure more reliable than many commercial alternatives.

Git

Version control before Git was either centralized and slow or proprietary. Linus created Git specifically to handle the scale and speed of Linux kernel development. **The community turned distributed version control into the default way for modern software development. Almost every modern development workflow sits on top of Git.** Its impact is less about any single feature and more about making collaboration across continents feel natural.

Apache HTTP Server

In the mid-1990s the leading web server was still proprietary. **Developers improved the NCSA code and released it under an open license.** Apache became the dominant web server for years. **Its community later formed the Apache Software Foundation and created a governance model many projects still follow.** It showed that critical internet infrastructure could be run by a neutral, multi-company community.

> *NCSA HTTPd was one of the earliest web server programs, originally developed at the National Center for Supercomputing Applications (NCSA) at the University of Illinois.*

Blender

Professional 3D animation and VFX tools used to cost thousands of dollars per user. Blender started as an in-house studio tool, **nearly died when its company collapsed, and was saved by a community crowdfunding campaign that bought the rights and released it under the GPL.** The community then did something unusual: it used the software to make entire open movies (Elephants Dream, Big Buck Bunny, Sintel, and others), which both stress-tested the tool and proved it could produce real production work. **Today Blender is used by indie studios, major animation houses, game developers, and freelancers.** It changed the economics of 3D creation and **demonstrated that an artist-driven open-source project could reach professional quality without locking users into subscriptions.**

Firefox

When Internet Explorer held a near-monopoly in the early 2000s, the web risked becoming a single-vendor platform. Mozilla took the open-sourced Netscape code, rebuilt it into Firefox, and shipped a browser that emphasized standards, privacy, and user control. The community around it helped it to reach **roughly a third of the market at its peak**, and forced Microsoft to improve Internet Explorer. Even though Firefox's market share is much smaller today, **its lasting effect is the continued existence of an independent rendering engine (Gecko) and a persistent counterweight to the Chromium monoculture.**

MuseScore and Audacity

These two tools did for music creation and audio editing what Blender did for 3D. MuseScore **gave composers and educators a free, high-quality scorewriter that could compete with expensive commercial packages**; **Audacity became the default free audio editor for podcasters, musicians, and students.** Both projects grew large user communities that contributed translations, plugins, and documentation. Their stories are also more complicated: both were later acquired by the same commercial group (Muse Group), which triggered debates about how open-source projects should handle ownership changes and monetization. **The lasting effect is still clear, millions of people who would never have paid for professional music or audio software now have capable tools, and the bar for entry into those creative fields is dramatically lower.**

Together these projects show the different ways open source can reshape a field: by providing critical infrastructure (Linux, Apache, Git), by breaking price barriers in creative industries (Blender, MuseScore, Audacity), or by defending openness against monopoly (Firefox). In each case the software mattered, but the community that formed around it determined whether the change stuck.

The lasting impact on the software industry

Open source permanently changed how software is built and who gets to build it. It turned collaboration across companies and continents into the default rather than the exception, lowered the barrier for individuals and small teams to create professional-grade tools, and forced proprietary vendors to compete on quality and openness instead of lock-in alone. Foundations, distributed version control, and community-driven development models that began in open source now shape even the largest commercial products. The result is an industry where critical infrastructure is often maintained collectively, creative tools are accessible without expensive licenses, and the expectation of transparency and shared ownership has become part of the culture of software itself.

bonus

> *useful GitHub repository:*
> *https://github.com/Furthir/awesome-useful-projects*